

Reviewer Pack — Minimal Geometric Quantization Dimension and Frege Proof Dep...

Entry 06f8cf7afd9d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 15:50:47 UTC

Minimal Geometric Quantization Dimension and Frege Proof Depth Inequality

Entry ID: 06f8cf7afd9d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 15:50:47 UTC

1. Conjecture

Field A (mathematical branch): Geometric Quantization Theory **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

For every Boolean formula φ with n variables, the minimal geometric quantization dimension ($gqd(\varphi)$) of its quantum state representation is linearly correlated with its Frege proof depth $w_F(\varphi)$, such that $gqd(\varphi) = \Omega(w_F(\varphi))$.

Rationale (proposer's reasoning):

Geometric quantization theory maps classical mechanics to quantum mechanics, providing a bridge between geometry and algebra. By associating the geometric quantization dimension with the Frege proof depth, we may uncover hidden relationships between geometric structures and computational complexity.

Taxonomy category: quantum_computational_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1c545ec2e5fed03a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all Boolean formulas φ with n variables, $gqd(\varphi) / w_F(\varphi) > 1.0$ AND $|gqd(\varphi) - w_F(\varphi)| \leq 3$ across at least 24 out of 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
from itertools import combinations, permutations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def quantum_state_dimension(formula):
        if formula == '0' or formula == '1':
            return 1
        elif formula.startswith('¬'):
            return quantum_state_dimension(formula[1:])
        else:
            op_index = next(i for i, char in enumerate(formula) if char in ('^', 'v'))
            left_dim = quantum_state_dimension(formula[:op_index])
            right_dim = quantum_state_dimension(formula[op_index + 1:])
            return left_dim * right_dim

    def frege_proof_depth(formula):
        if formula == '0' or formula == '1':
            return 1
        elif formula.startswith('¬'):
            return 1 + frege_proof_depth(formula[1:])
        else:
            op_index = next(i for i, char in enumerate(formula) if char in ('^', 'v'))
            left_depth = frege_proof_depth(formula[:op_index])
            right_depth = frege_proof_depth(formula[op_index + 1:])
            return 1 + max(left_depth, right_depth)

    def generate_random_formula(n):
        if n == 0:
            return random.choice(['0', '1'])
        else:
            variables = list(range(1, n + 1))
            formula = []
            for _ in range(random.randint(1, n)):
                op = random.choice(['^', 'v'])
                args = [generate_random_formula(n - 1) for _ in range(2)]
```

```

        formula.append(f'({op.join(args)})')
    return '(' + ' '.join(formula) + ')'

results = []
for n in [5, 10, 15, 20, 30, 40]:
    for _ in range(5):
        formula = generate_random_formula(n)
        gqd = quantum_state_dimension(formula)
        w_F = frege_proof_depth(formula)
        results.append((gqd, w_F))

if not results:
    return {
        "metric_name": "GQD / w_F",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "No formulas generated"
    }

gqd_values = [gqd for gqd, _ in results]
w_F_values = [w_F for _, w_F in results]
mean_gqd = sum(gqd_values) / len(gqd_values)
mean_w_F = sum(w_F_values) / len(w_F_values)
ratio = mean_gqd / mean_w_F
diff = abs(mean_gqd - mean_w_F)

return {
    "metric_name": "GQD / w_F",
    "metric_value": ratio,
    "instances_tested": len(results),
    "n_max": max(n for _, n in [(gqd, n) for gqd, _ in results]),
    "conjecture_holds": ratio > 1.0 and diff <= 3,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        support_fraction = len([result for result in results if result["conjecture_holds"]]) / len(results)
        mean_ratio = sum(result["metric_value"] for result in results) / len(results)
        std_dev = (sum((result["metric_value"] - mean_ratio)**2 for result in results) / len(results))**0.5
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:

```

```
print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4232db94.py", line 94, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4232db94.py", line 58, in run_trial
    gqd = quantum_state_dimension(formula)
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4232db94.py", line 27, in quantum_state
    left_dim = quantum_state_dimension(formula[:op_index])
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4232db94.py", line 26, in quantum_state
    op_index = next(i for i, char in enumerate(formula) if char in ('^', 'v'))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

StopIteration

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified according to the pre-registered criteria. | next: Investigate and fix the crash in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15013
2	propose	ollama_remote	glm4:latest	0	0	12450
3	preregistration	ollama_remote	glm4:latest	0	0	11266
4	novelty	ollama_remote	glm4:latest	0	0	8844
5	novelty	ollama_remote	glm4:latest	0	0	8460
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15127
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10746
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11535

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11727
10	judge	ollama_remote	glm4:latest	0	0	22370

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 127536 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/06f8cf7afd9d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/06f8cf7afd9d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/06f8cf7afd9d.tar.gz (if generated)